VANATVAM

Booking Management System

GCP Deployment Guide

Step-by-Step Google Cloud Platform Setup

Version: May 2026 | Confidential & Internal Use Only

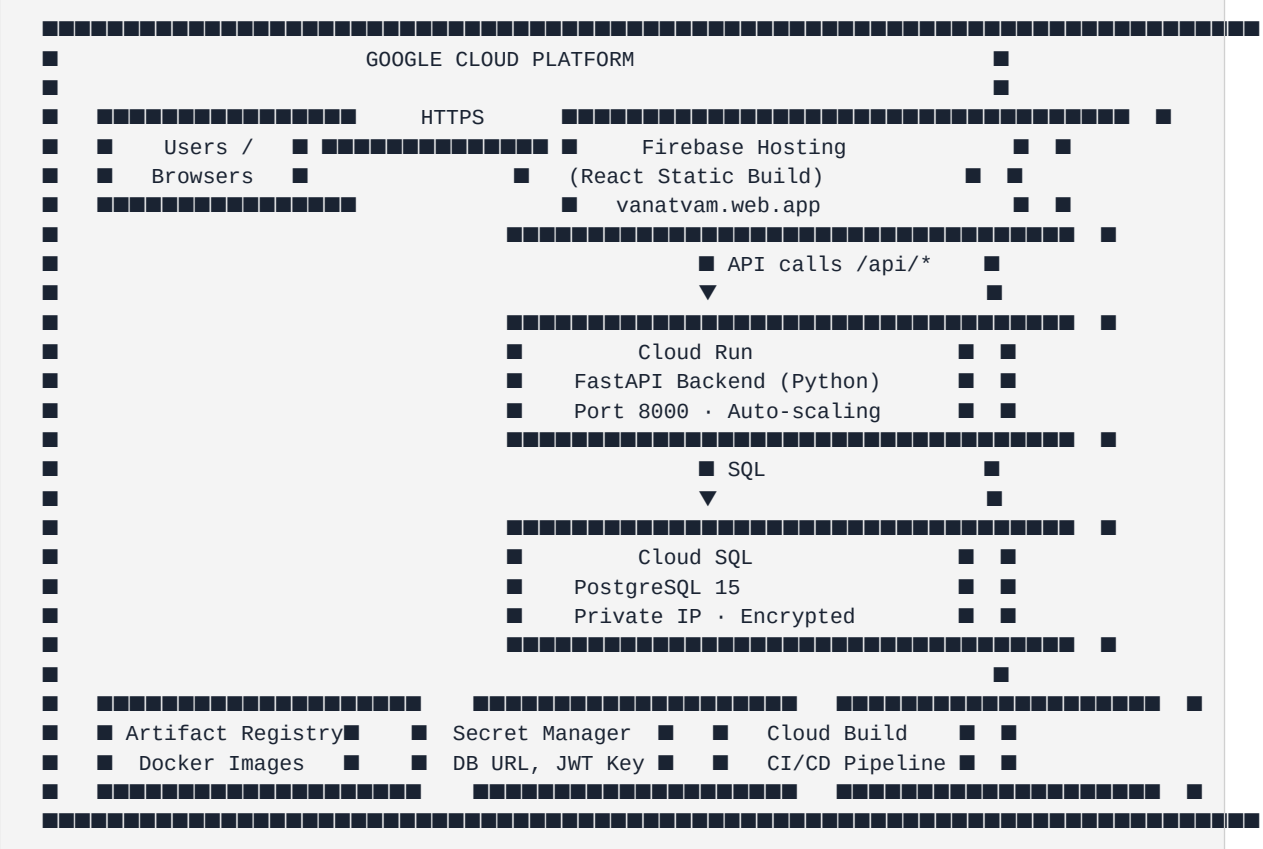
vanatvam-booking-app.web.app

■ Vanatvam — Google Cloud Platform Deployment Guide

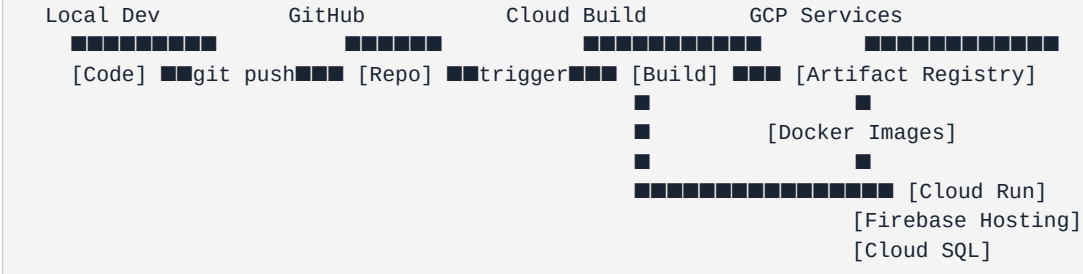
> ***Stack:*** *React (TypeScript) Frontend · FastAPI (Python) Backend · PostgreSQL Database*

> ***GCP Services Used:*** *Cloud Run · Cloud SQL · Firebase Hosting · Artifact Registry · Secret Manager · Cloud Build*

■ Architecture Overview



■ Deployment Flow



■ Prerequisites

Before starting, ensure you have the following installed on your Mac:

Google Cloud SDK	`brew install google-cloud-sdk`	GCP CLI
Docker Desktop	docker.com/products/docker-desktop	Container builds
Node.js 18+	`brew install node`	Frontend build
Python 3.11+	Already installed	Backend

■ Step 1 — Set Up Google Cloud Project

1.1 Create a New GCP Project

1. Go to console.cloud.google.com
2. Click the **project dropdown** at the top → **New Project**
3. Fill in:

- **Project name:** `vanatvam-booking`

- **Project ID:** `vanatvam-booking` *(note this down — you'll use it everywhere)*

4. Click **Create**

1.2 Enable Billing

1. In the GCP Console, go to **Billing** in the left sidebar
2. Link a billing account to your project
3. *(Cloud Run has a generous free tier — 2 million requests/month free)*

1.3 Enable Required APIs

Open **Cloud Shell** (terminal icon in top-right of GCP Console) and run:

```
# Set your project
gcloud config set project vanatvam-booking

# Enable all required APIs
gcloud services enable \
  run.googleapis.com \
  sql-component.googleapis.com \
  sqladmin.googleapis.com \
  artifactregistry.googleapis.com \
  cloudbuild.googleapis.com \
  secretmanager.googleapis.com \
  firebase.googleapis.com \
  firebasehosting.googleapis.com
```

> ■ *This takes about 1–2 minutes. You'll see* `Operation finished successfully` *when done.*

■■ Step 2 — Set Up Cloud SQL (PostgreSQL)

2.1 Create the Database Instance

In Cloud Shell:

```
gcloud sql instances create vanatvam-db \
  --database-version=POSTGRES_15 \
  --tier=db-f1-micro \
  --region=asia-south1 \
  --storage-auto-increase \
  --storage-size=10GB \
  --backup-start-time=02:00 \
  --availability-type=zonal
```

> ■ *This takes **5–10 minutes**. The `db-f1-micro` tier is the cheapest (~$7/month).*

2.2 Create the Database and User

```
# Create the database
gcloud sql databases create vanatvam \
  --instance=vanatvam-db

# Create a dedicated user (replace YOUR_STRONG_PASSWORD)
gcloud sql users create vanatvam_user \
  --instance=vanatvam-db \
  --password=YOUR_STRONG_PASSWORD
```

2.3 Get the Connection Name

```
gcloud sql instances describe vanatvam-db \
  --format="value(connectionName)"
```

> ■ ***Save this output** — it looks like: `vanatvam-booking:asia-south1:vanatvam-db`*

■ Step 3 — Store Secrets in Secret Manager

Never put passwords in environment variables directly. Use Secret Manager:

```
# Database URL secret
echo -n "postgresql+psycopg2://vanatvam_user:YOUR_STRONG_PASSWORD@/vanatvam?host=/cloudsql/vanatvam-booking
  gcloud secrets create DATABASE_URL --data-file=-

# JWT Secret Key (generate a strong random key)
python3 -c "import secrets; print(secrets.token_hex(32))" | \
  gcloud secrets create JWT_SECRET_KEY --data-file=-

# Email settings (if using email features)
echo -n "your-email@gmail.com" | gcloud secrets create EMAIL_FROM --data-file=-
echo -n "your-app-password" | gcloud secrets create EMAIL_PASSWORD --data-file=-
```

■ Step 4 — Create Docker Files

4.1 Backend Dockerfile

Create `/Users/rohithkumar/Documents/MySites/Vanatvam-Booking/backend/Dockerfile`:

```
FROM python:3.11-slim

WORKDIR /app

# Install system dependencies for psycopg2
RUN apt-get update && apt-get install -y \
    gcc \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*

# Copy and install Python dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

# Expose port
EXPOSE 8000

# Run the application
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

4.2 Frontend Dockerfile (for local testing only)

Create `/Users/rohithkumar/Documents/MySites/Vanatvam-Booking/frontend/Dockerfile`:

```
FROM node:18-alpine AS builder

WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=builder /app/build /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

4.3 Frontend nginx.conf

Create `/Users/rohithkumar/Documents/MySites/Vanatvam-Booking/frontend/nginx.conf`:

```
server {
    listen 80;
    server_name _;
    root /usr/share/nginx/html;
    index index.html;

    # Handle React Router (SPA)
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Cache static assets
    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
}
```

■ Step 5 — Set Up Artifact Registry

```
# Create a Docker repository
gcloud artifacts repositories create vanatvam-repo \
  --repository-format=docker \
  --location=asia-south1 \
  --description="Vanatvam Docker images"

# Configure Docker to authenticate with GCP
gcloud auth configure-docker asia-south1-docker.pkg.dev
```

■■ Step 6 — Build & Push Docker Images

6.1 Build and Push Backend Image

```
# Navigate to backend directory
cd /Users/rohithkumar/Documents/MySites/Vanatvam-Booking/backend

# Build the image
docker build -t asia-south1-docker.pkg.dev/vanatvam-booking/vanatvam-repo/backend:latest .

# Push to Artifact Registry
docker push asia-south1-docker.pkg.dev/vanatvam-booking/vanatvam-repo/backend:latest
```

■■ Step 7 — Deploy Backend to Cloud Run

7.1 Grant Cloud Run Access to Secrets

```
# Get the Cloud Run service account
PROJECT_NUMBER=$(gcloud projects describe vanatvam-booking --format="value(projectNumber)")

# Grant Secret Manager access
gcloud projects add-iam-policy-binding vanatvam-booking \
  --member="serviceAccount:${PROJECT_NUMBER}-compute@developer.gserviceaccount.com" \
  --role="roles/secretmanager.secretAccessor"

# Grant Cloud SQL access
gcloud projects add-iam-policy-binding vanatvam-booking \
  --member="serviceAccount:${PROJECT_NUMBER}-compute@developer.gserviceaccount.com" \
  --role="roles/cloudsql.client"
```

7.2 Deploy the Backend Service

```
gcloud run deploy vanatvam-backend \
  --image=asia-south1-docker.pkg.dev/vanatvam-booking/vanatvam-repo/backend:latest \
  --region=asia-south1 \
  --platform=managed \
  --allow-unauthenticated \
  --port=8000 \
  --memory=512Mi \
  --cpu=1 \
  --min-instances=0 \
  --max-instances=10 \
  --add-cloudsql-instances=vanatvam-booking:asia-south1:vanatvam-db \
  --set-secrets="DATABASE_URL=DATABASE_URL:latest,JWT_SECRET_KEY=JWT_SECRET_KEY:latest"
```

7.3 Get the Backend URL

```
gcloud run services describe vanatvam-backend \
  --region=asia-south1 \
  --format="value(status.url)"
```

> ■ ***Save this URL*** *— it looks like:* `https://vanatvam-backend-xxxxxxxx-el.a.run.app`

■ Step 8 — Deploy Frontend to Firebase Hosting

Firebase Hosting is the best option for React SPAs — it's fast, free, and has a global CDN.

8.1 Install Firebase CLI

```
npm install -g firebase-tools
```

8.2 Login and Initialize Firebase

```
# Login to Firebase
firebase login

# Navigate to frontend directory
cd /Users/rohithkumar/Documents/MySites/Vanatvam-Booking/frontend

# Initialize Firebase Hosting
firebase init hosting
```

When prompted:

- **Which Firebase project?** → Select `vanatvam-booking`
- **Public directory?** → Type `build`
- **Single-page app?** → `Yes`
- **Automatic builds with GitHub?** → `No` (we'll do manual for now)

8.3 Update Frontend API URL

Update `/Users/rohithkumar/Documents/MySites/Vanatvam-Booking/frontend/.env`:

```
REACT_APP_API_URL=https://vanatvam-backend-xxxxxxxx-el.a.run.app
```

> ■■ *Replace* `xxxxxxxx` *with your actual Cloud Run URL from Step 7.3*

8.4 Update CORS in Backend

Update `backend/main.py` to allow your Firebase domain:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:3000",                    # Local dev
        "https://vanatvam-booking.web.app",         # Firebase Hosting
        "https://vanatvam-booking.firebaseapp.com", # Firebase alternate
    ],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

8.5 Build and Deploy Frontend

```
cd /Users/rohithkumar/Documents/MySites/Vanatvam-Booking/frontend

# Build the production React app
npm run build

# Deploy to Firebase Hosting
firebase deploy --only hosting
```

> ■ *Your app will be live at:* `https://vanatvam-booking.web.app`

■ Step 9 — Set Up CI/CD with Cloud Build (Optional but Recommended)

9.1 Create Cloud Build Configuration

Create `/Users/rohithkumar/Documents/MySites/Vanatvam-Booking/cloudbuild.yaml`:

```
steps:
  # Step 1: Build Backend Docker Image
  - name: 'gcr.io/cloud-builders/docker'
    args:
      - 'build'
      - '-t'
      - 'asia-south1-docker.pkg.dev/$PROJECT_ID/vanatvam-repo/backend:$COMMIT_SHA'
      - './backend'
    id: 'build-backend'

  # Step 2: Push Backend Image
  - name: 'gcr.io/cloud-builders/docker'
    args:
      - 'push'
      - 'asia-south1-docker.pkg.dev/$PROJECT_ID/vanatvam-repo/backend:$COMMIT_SHA'
    id: 'push-backend'
    waitFor: ['build-backend']

  # Step 3: Deploy Backend to Cloud Run
  - name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'
    entrypoint: gcloud
    args:
      - 'run'
      - 'deploy'
      - 'vanatvam-backend'
      - '--image=asia-south1-docker.pkg.dev/$PROJECT_ID/vanatvam-repo/backend:$COMMIT_SHA'
      - '--region=asia-south1'
      - '--platform=managed'
    id: 'deploy-backend'
    waitFor: ['push-backend']

  # Step 4: Build Frontend
  - name: 'node:18'
    entrypoint: 'bash'
    args:
      - '-c'
      - 'cd frontend && npm ci && npm run build'
    id: 'build-frontend'

  # Step 5: Deploy Frontend to Firebase
  - name: 'node:18'
    entrypoint: 'bash'
    args:
      - '-c'
      - 'npm install -g firebase-tools && cd frontend && firebase deploy --only hosting --token $$FIREBASE_
    secretEnv: ['FIREBASE_TOKEN']
    id: 'deploy-frontend'
    waitFor: ['build-frontend']

availableSecrets:
  secretManager:
    - versionName: projects/$PROJECT_ID/secrets/FIREBASE_TOKEN/versions/latest
      env: 'FIREBASE_TOKEN'

options:
  logging: CLOUD_LOGGING_ONLY
```

9.2 Connect GitHub Repository

1. In GCP Console → **Cloud Build** → **Triggers**
2. Click **Connect Repository**
3. Select **GitHub** → Authenticate

4. Select your `Vanatvam-Booking` repository
5. Click **Create a trigger**:

- **Name:** `deploy-on-push`
- **Event:** Push to branch
- **Branch:** `^main$`
- **Build config:** `cloudbuild.yaml`

6. Click **Create**

■ Step 10 — Custom Domain (Optional)

10.1 Add Custom Domain to Firebase Hosting

1. In Firebase Console → **Hosting** → **Add custom domain**
2. Enter your domain (e.g., `vanatvam.com`)
3. Follow the DNS verification steps
4. Add the provided DNS records to your domain registrar

10.2 Add Custom Domain to Cloud Run

```
gcloud run domain-mappings create \
  --service=vanatvam-backend \
  --domain=api.vanatvam.com \
  --region=asia-south1
```

■ Estimated Monthly Costs

Cloud Run (Backend)	0–2M requests/month	**Free**
Cloud SQL (PostgreSQL)	db-f1-micro	~$7–10/month
Firebase Hosting	Spark plan (10GB/month)	**Free**
Artifact Registry	First 0.5GB	**Free**
Secret Manager	First 6 secrets	**Free**
Cloud Build	First 120 min/day	**Free**
Total		**~$7–10/month**

■ Step 11 — Verify Deployment

11.1 Test Backend API

```
# Get your Cloud Run URL
BACKEND_URL=$(gcloud run services describe vanatvam-backend \
  --region=asia-south1 --format="value(status.url)")

# Test the health endpoint
curl $BACKEND_URL/
# Expected: {"message": "Vanatvam API is running"}

# Test the API docs
echo "API Docs: $BACKEND_URL/docs"
```

11.2 Test Frontend

Open your browser and navigate to:

- `https://vanatvam-booking.web.app` — Login page should appear
- Try registering a new account
- Try logging in as admin

■■ Troubleshooting

■ Cloud Run: "Container failed to start"

```
# View Cloud Run logs
gcloud run services logs read vanatvam-backend \
  --region=asia-south1 \
  --limit=50
```

Common causes:

- Missing environment variables → Check Secret Manager secrets are correctly named
- Database connection failed → Verify Cloud SQL instance name in `--add-cloudsql-instances`
- Port mismatch → Ensure Dockerfile exposes port `8000` and Cloud Run uses `--port=8000`

■ Cloud SQL: "Connection refused"

```
# Check Cloud SQL instance status
gcloud sql instances describe vanatvam-db --format="value(state)"
# Should return: RUNNABLE
```

Make sure the DATABASE_URL uses the Unix socket format for Cloud Run:

```
postgresql+psycopg2://vanatvam_user:PASSWORD@/vanatvam?host=/cloudsql/PROJECT:REGION:INSTANCE
```

■ Frontend: "CORS error" in browser console

- Ensure the backend's `allow_origins` list includes your Firebase Hosting URL
- Redeploy backend after updating CORS settings
- Clear browser cache and try again

■ Firebase: "Permission denied"

```
# Re-authenticate Firebase
firebase logout
firebase login
```

■ Monitoring & Logs

View Backend Logs in Real-time

```
gcloud run services logs tail vanatvam-backend --region=asia-south1
```

View Cloud SQL Logs

In GCP Console → **Cloud SQL** → **vanatvam-db** → **Operations & logs**

Set Up Uptime Monitoring

1. GCP Console → **Monitoring** → **Uptime checks**
2. Click **Create uptime check**
3. Target: your Cloud Run URL
4. Period: every 5 minutes
5. Add email alert notification

■ Updating the Application

Update Backend Only

```
cd /Users/rohithkumar/Documents/MySites/Vanatvam-Booking/backend

# Build new image
docker build -t asia-south1-docker.pkg.dev/vanatvam-booking/vanatvam-repo/backend:latest .
docker push asia-south1-docker.pkg.dev/vanatvam-booking/vanatvam-repo/backend:latest

# Deploy new version (zero downtime)
gcloud run deploy vanatvam-backend \
  --image=asia-south1-docker.pkg.dev/vanatvam-booking/vanatvam-repo/backend:latest \
  --region=asia-south1
```

Update Frontend Only

```
cd /Users/rohithkumar/Documents/MySites/Vanatvam-Booking/frontend

npm run build
firebase deploy --only hosting
```

■ Final File Structure After Setup

```
Vanatvam-Booking/
■■■ backend/
■   ■■■ Dockerfile          ← NEW: Docker config for backend
■   ■■■ main.py             ← Updated CORS origins
■   ■■■ requirements.txt
■   ■■■ ...
■■■ frontend/
■   ■■■ Dockerfile          ← NEW: Docker config for frontend
■   ■■■ nginx.conf          ← NEW: Nginx config for SPA routing
■   ■■■ .env                ← Updated with Cloud Run API URL
■   ■■■ .firebaserc         ← NEW: Firebase project config
■   ■■■ firebase.json       ← NEW: Firebase hosting config
■   ■■■ ...
■■■ cloudbuild.yaml         ← NEW: CI/CD pipeline config
■■■ GCP_DEPLOYMENT_GUIDE.md ← This file
```

■ Deployment Checklist

- [ ] GCP project created and billing enabled
- [ ] All required APIs enabled
- [ ] Cloud SQL instance created and running
- [ ] Database and user created
- [ ] Secrets stored in Secret Manager
- [ ] Backend Dockerfile created
- [ ] Backend Docker image built and pushed to Artifact Registry
- [ ] Backend deployed to Cloud Run
- [ ] Frontend `.env` updated with Cloud Run URL
- [ ] Backend CORS updated with Firebase Hosting URL
- [ ] Firebase CLI installed and initialized
- [ ] Frontend built (`npm run build`)
- [ ] Frontend deployed to Firebase Hosting
- [ ] Backend API health check passing
- [ ] Frontend login page loading correctly
- [ ] End-to-end login flow tested

Guide created for Vanatvam Booking Management System · GCP Deployment · February 2026